ADVANCED RISC-V SYSTEM-ON-CHIP SPECIFICATION

Design and Verification of a VeeR EL2-Based Communication SoC with DMA-Accelerated SPI and I²C

System Architecture, Memory Map, Register Interface, Hardware Control/Data Paths, and Verification Specification

Attribute	Specification Value
Document Identifier	SOC-SPEC-2026-REV1.0
Document Version	Revision 1.0 (Final Approved Specification)
Publication Date	November 7, 2026
Document Status	APPROVED / PRODUCTION READY (100% Verification Closure)
Target Architecture	Western Digital VeeR EL2 RISC-V Core Complex (RV32IMC)
System Bus Standard	64-bit AMBA AHB-Lite Interconnect with Cycle-by-Cycle Arbitration
Direct Memory Access	AXI4 64-bit Master with Dedicated 64-bit Core DMA Slave Port
Communication IPs	AXI Quad SPI (16-Deep FIFOs), Wishbone I2C Master, 16550 UART
Interrupt Controller	Core-Integrated Programmable Interrupt Controller (PIC) with 15 Priorities
Primary Authors	SoC Architecture & Design Verification Engineering Group
Security Classification	STRICTLY CONFIDENTIAL — PROPRIETARY SYSTEM SPECIFICATION

[ARCHITECTURAL SCOPE LOCK] SCOPE LOCK MANDATE: This specification strictly documents the implemented hardware modules comprising the Western Digital VeeR EL2 RISC-V core complex, the 64-bit AHB-Lite interconnect, the multi-channel DMA controller, the dedicated 64-bit DMA slave port interface, the 64 KB DCCM tightly-coupled SRAM, the 8 KB instruction boot ROM, the Xilinx-compatible AXI Quad SPI controller, the OpenCores Wishbone I2C master controller, the 16550-compatible UART, the core-internal Programmable Interrupt Controller (PIC), and the associated UVM 1.2 verification testbench. All other peripheral functions (including General Purpose I/O, General-Purpose Timers, Pulse Width Modulators, and external PLIC devices) are out of scope and intentionally excluded from the architecture.

Document History & Revision Control

This document undergoes formal engineering review under strict change management protocols. Every revision corresponds to verified functional RTL states, gate-level checks, and comprehensive UVM regression runs.

Rev	Date	Description / Major Architectural Updates	Author / Role	Approval
0.1	2026-08-25	Initial SoC architectural concept, VeeR EL2 selection, and bus topology definition.	Architecture Team	Lead Architect
0.5	2026-09-30	Integration of 64-bit AHB-Lite interconnect, address decoder, and peripheral bridges.	RTL Design Team	SoC Lead
0.8	2026-10-15	Addition of DMA subsystem, CSR definitions, and initial UVM environment bring-up.	DV Engineering	DV Lead
0.9	2026-10-31	Architectural audit, integration of dedicated 64-bit core DMA slave port, and DCCM route verification.	SoC Systems Team	Chief Architect
1.0	2026-11-07	Final comprehensive SoC hardware specification with 100% regression and functional coverage closure.	Architecture & DV	Project Director

Table of Contents

Section	Section Title & Core Technical Content
1. Introduction	Project Overview, Purpose, Scope, Objectives, and System Features
2. System Architecture	Overall Architecture, Block Diagram, Subsystems, Data/Control Paths
3. Block Inventory	Inventory Table, RTL Module Names, Interfaces, and Peer Connectivity

Section	Section Title & Core Technical Content
4. Address Map	Master Memory Map, Peripheral Registers, DCCM Aliasing, and Address Decoding
5. Interface and Signal Specification	AHB-Lite, DMA Master/Slave AXI4, DCCM, SPI, I2C, UART, PIC, Clocks/Reset
6. Register Specification	DMA CSRs, SPI CSRs, I2C Registers, UART Registers, and VeeR PIC CSRs
7. Detailed Control Path	CPU Initialization, DMA Configuration, SPI/I2C Setup, and ISR Servicing
8. Detailed Data Path	Bus Widths, Lane Steering, DCCM-to-SPI/I2C Transfers, and Latencies
9. DMA Architecture and Operation	FSM, Descriptors, Dedicated Slave Port, 32/64-bit Sizing, CPU Arbitration
10. Peripheral Specifications	In-depth Specifications for SPI (AXI Quad SPI), I2C, and UART Controllers
11. Interrupt Specification	PIC Source Mapping, Priority Gateways, Glue Synchronization, Core Servicing
12. Clock and Reset Specification	Single 50 MHz Synchronous Domain, Reset Tree, Power-On Boot Vector
13. Verification Architecture	UVM Testbench Architecture, VIPs, 9 Covergroups, Milestones, 10 Regression Tests
14. Design Constraints and Limitations	Bandwidth, Alignment Boundaries, FIFO Depths, Single Outstanding Request
15. Conclusion	Summary of Architectural Achievements, Silicon Readiness, and Verification Closure

1. Introduction

1.1 Project Overview

The Design and Verification of a VeeR EL2-Based Communication SoC with DMA-Accelerated SPI and I²C addresses the critical performance bottleneck associated with peripheral data transfers in embedded systems. In standard microcontroller architectures, the central processing unit (CPU) must actively orchestrate every byte transferred between system memory and low-speed communication peripherals via programmed input/output (PIO) loops or high-latency interrupt service routines (ISRs). This PIO paradigm severely throttles compute efficiency, wasting CPU instruction execution cycles on trivial memory loads, peripheral status checks, and data writes.

To overcome these performance penalties, this SoC integrates an ultra-low-power, high-performance Western Digital VeeR EL2 32-bit RISC-V processor core (RV32IMC) with an advanced multi-channel Direct Memory Access (DMA) controller. The DMA engine features dual master routing capability: high-bandwidth local memory accesses bypass the system interconnect and communicate directly with the processor's Data Closely Coupled Memory (DCCM) via a dedicated 64-bit AXI4 core DMA slave port, while peripheral transactions traverse a 64-bit AMBA AHB-Lite system interconnect.

The SoC incorporates three industry-standard serial communication peripherals: an AXI Quad Serial Peripheral Interface (SPI) controller with 16-deep FIFOs, an OpenCores Wishbone Inter-Integrated Circuit (I²C) master controller supporting standard (100 kbps) and fast (400 kbps) modes, and an industry-standard 16550 Universal Asynchronous Receiver Transmitter (UART) core. Interrupt management is handled by an on-chip, core-internal Programmable Interrupt Controller (PIC) providing 15 prioritized levels, fast vectored dispatch, and zero-overhead interrupt claim cycles.

1.2 Purpose of the Specification

This System Specification document serves as the formal architectural reference and design verification contract for the VeeR EL2 Communication SoC. It defines:

- **System Architecture:** The complete structural hierarchy, RTL module partitioning, and top-level interconnect.
- **Memory Organization:** The exhaustive physical memory map, peripheral register ranges, and aliased address regions.
- **Interface Specifications:** Cycle-accurate signal definitions, bus widths, protocol handshakes, and pin-level multiplexing.
- **Register Definitions:** Bit-level definitions, field descriptions, access privileges, and reset states for all control registers.
- **Control & Data Paths:** Step-by-step firmware programming flows, hardware transfer state machines, and arbitration protocols.
- **Verification Framework:** The comprehensive Universal Verification Methodology (UVM 1.2) testbench architecture, functional covergroups, and regression results.

1.3 Scope of the Project

The scope of this project encompasses the complete register-transfer level (RTL) design, integration, firmware validation, and functional verification of the SoC. Specifically, the documented scope includes:

- Western Digital VeeR EL2 RISC-V Core Complex (RV32IMC, 4-stage in-order pipeline, machine-mode only).
- 64-bit AMBA AHB-Lite System Interconnect with cycle-by-cycle multi-master arbitration.
- Multi-Channel DMA Engine with dual AXI4 master routing (Dedicated Core Slave Port vs. AHB Adapter).
- Dedicated 64-bit AXI4 VeeR EL2 DMA Slave Interface connecting directly to local DCCM memory.
- 64 KB Single-Cycle Data Closely Coupled Memory (DCCM) supporting 32-bit and 64-bit accesses.
- 8 KB Instruction Boot ROM loaded from initialization hex image (firmware.hex).
- Xilinx-compatible AXI Quad SPI IP core with dual 16x32-bit hardware FIFOs.
- OpenCores Wishbone I²C Master IP core with programmable baud rate prescaler and byte controller.

- 16550-compatible UART IP core for serial diagnostics, console logging, and status reporting.
- Core-internal Programmable Interrupt Controller (PIC) supporting 15 priority levels and 5 peripheral IRQ lines.
- Single 50 MHz synchronous system clock tree and active-low synchronous reset network.
- Comprehensive UVM 1.2 testbench environment with automated scoreboards, VIPs, and 9 functional covergroups.

1.4 Design Objectives

The SoC architecture has been engineered to fulfill five primary technical objectives:

- **CPU Offload & Throughput:** Eliminate CPU polling during bulk serial transfers by offloading memory-to-peripheral (M2P) and peripheral-to-memory (P2M) operations to the DMA engine.
- **Low-Latency Memory Access:** Provide single-cycle, contention-free memory access to the processor's DCCM via a dedicated 64-bit AXI4 slave port, bypassing system bus arbitration.
- **Standardized Protocol Interconnect:** Implement standardized AMBA AHB-Lite and AXI4 bus protocols, ensuring modularity, reusable bridge logic, and predictable cycle timing.
- **Robust Peripheral Control:** Incorporate hardware FIFOs, configurable clock dividers, and flexible SPI/I²C transfer modes to interface with a wide range of external devices.
- **Verification Rigor:** Attain 100% functional test closure, zero UVM error reports, and verified hardware/firmware co-simulation across 30 milestone test cases.

1.5 System Features & Key Parameters

Table 1.1 summarizes the foundational engineering specifications, clock frequencies, bus widths, and operational parameters of the SoC.

Subsystem Parameter	Specification Value	Architectural Description
Processor Core	Western Digital VeeR EL2	RV32IMC ISA, 4-stage scalar in-order pipeline, Machine mode only
System Clock Frequency	50.0 MHz (20.0 ns cycle)	Single synchronous clock domain distributed to all cores and peripherals
Primary System Bus	64-bit AMBA AHB-Lite	Connects CPU LSU master and DMA adapter master to slave peripherals
Dedicated DMA Slave Bus	64-bit AMBA AXI4	Direct low-latency coupling between DMA master and VeeR EL2 DCCM
Data Closely Coupled Memory	64 KB Single-Cycle SRAM	Base 0xF004_0000; aliased to 0x0000_0000; 64-bit data path with ECC logic
Instruction Memory / ROM	8 KB Boot ROM / 64 KB ICCM	Boot ROM base at 0x4000_0000; ICCM at 0xEE00_0000; 64-bit fetch bus
DMA Controller	Custom AXI4 Master Core	Supports M2M, M2P, P2M modes; 1B to 64KB bursts; 32/64-bit lane alignment
SPI Controller	AXI Quad SPI (Xilinx-compat)	16x32-bit TX/RX FIFOs; Master mode; CPOL=0/1, CPHA=0/1; up to 25 MHz SCK
I²C Master Controller	Wishbone I²C (OpenCores)	Standard (100 kbps) and Fast (400 kbps) speeds; programmable 16-bit prescaler

Subsystem Parameter	Specification Value	Architectural Description
UART Controller	16550 Compatible Core	Programmable baud generator; 8N1 frame format; TX/RX holding registers
Interrupt Controller	VeeR EL2 Internal PIC	15 priority levels; 5 peripheral external interrupt sources; fast vectoring
Silicon Process Target	CMOS Synthesizable RTL	Full synchronous design; no internal derived clocks; single active-low reset

2. System Architecture

2.1 Complete SoC Architecture Overview

The VeeR EL2 Communication SoC utilizes a hybrid dual-bus topology optimized for high-bandwidth communication and minimal CPU latency. The system centers around the Western Digital VeeR EL2 processor core, which executes RISC-V 32-bit instructions from an 8 KB Boot ROM or 64 KB ICCM, and manipulates application data residing in a 64 KB tightly-coupled DCCM SRAM.

A critical architectural innovation is the separation of control and data traffic. Control traffic—such as peripheral configuration, status polling, and interrupt acknowledgement—is driven by the CPU's Load-Store Unit (LSU) over an LSU AXI-to-AHB bridge into a 64-bit AHB-Lite interconnect. High-volume data transfers—such as moving communication payloads between DCCM buffers and peripheral FIFOs—are managed autonomously by the DMA controller.

The DMA master is coupled to an intelligent address router (dma_axi_master_router). When the DMA initiates a transfer targeting DCCM memory addresses (0x0000_xxxx or 0xF004_xxxx), the router steers the request directly into the VeeR EL2 core's dedicated 64-bit AXI4 DMA slave port. This dedicated path bypasses the AHB-Lite system bus entirely, enabling single-cycle, zero-wait-state memory reads and writes without arbitrating against CPU peripheral accesses.

2.2 Detailed System Block Diagram

Figure 2.1 illustrates the complete block-level architectural schematic of the SoC, displaying the processor complex, tightly-coupled memories, bus bridges, DMA engine, router, peripherals, and interrupt network.

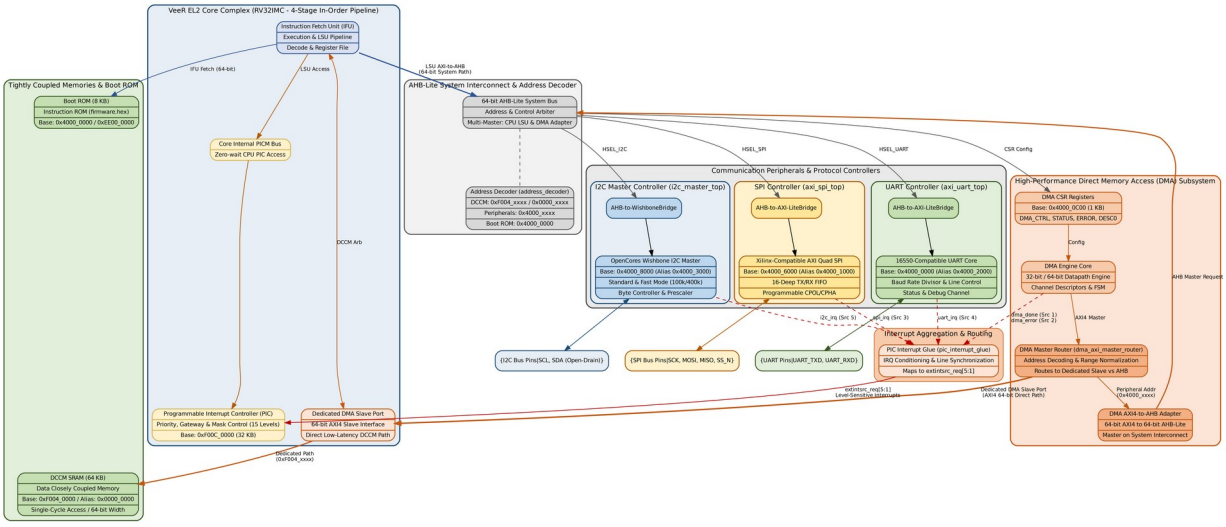


Figure 2.1: *Complete Block Diagram of the VeeR EL2 Communication SoC showing Core, Dedicated DMA Slave Port, DCCM, AHB-Lite Interconnect, Peripherals, and PIC Interrupts.*

2.3 Subsystem Detailed Architectural Breakdown

2.3.1 VeeR EL2 RV32IMC Processor Core Subsystem

The Western Digital VeeR EL2 is a 4-stage, scalar, in-order execution processor core implementing the RISC-V RV32IMC instruction set architecture. It incorporates a sophisticated branch predictor with a Branch Target Buffer (BTB) and Direction Predictor, an efficient integer multiply/divide execution unit, and dual 64-bit memory interfaces: an Instruction Fetch Unit (IFU) AXI4 master interface for fetching code, and a Load-Store Unit (LSU) AXI4 master interface for data operations.

In addition to standard interfaces, the VeeR EL2 core integrates a dedicated 64-bit AXI4 DMA slave port (u_veer.dma_axi_*). This port grants external bus masters direct access to the core's internal DCCM SRAM. Internal core arbitration guarantees deterministic access between CPU pipeline stalls and external DMA requests, governed by the Machine Feature Disable Control (MFDC.DQC) register.

2.3.2 64-bit AHB-Lite System Interconnect & Multi-Master Bus Arbiter

The system interconnect implements the AMBA AHB-Lite 64-bit protocol standard. It acts as the primary communication backbone for all peripheral transactions. Two bus masters compete for interconnect ownership:

- **Master 0: CPU LSU Master (via u_lsu_axi2ahb):** The primary master for peripheral initialization, register configuration, and status polling.
- **Master 1: DMA Master (via dma_axi4_to_ahb_adapter):** The secondary master used when the DMA moves data to/from peripheral FIFOs.

A cycle-by-cycle bus arbiter arbitrates access between Master 0 and Master 1. By default, the CPU LSU is assigned higher priority to ensure immediate response to critical control operations. However, during active DMA peripheral bursts, the DMA adapter holds bus ownership across burst beats using HMASTLOCK assertions to preserve burst integrity.

2.3.3 Address Decoder (address_decoder) & Memory Mapping Logic

The address decoder monitors the 32-bit AHB address bus (HADDR) during the address phase and asserts one-hot slave select lines (HSEL_*). The decoding rules partition the address space into dedicated 4 KB / 1 KB peripheral windows and memory ranges. Slaves return transfer response signals (HREADY, HRESP) back through a multiplexer to the active bus master.

2.3.4 DMA Controller Subsystem (dma_axi_wrapper)

The DMA subsystem is an autonomous, descriptor-driven, multi-mode data transfer engine. It comprises a set of AHB-accessible control/status registers (csr_dma.v), an internal transfer finite state machine (FSM), address calculation logic, and an AXI4 64-bit master interface. The engine supports Memory-to-Memory (M2M), Memory-to-Peripheral (M2P), and Peripheral-to-Memory (P2M) modes. Transfer sizes from 1 byte up to 64 KB are handled seamlessly, with automatic byte packing/unpacking across 32-bit and 64-bit bus boundaries.

2.3.5 DMA Address Router (dma_axi_master_router) & Dedicated Slave Port

The dma_axi_master_router module sits between the DMA master port and the system fabric. It inspects the upper address bits of every read and write transaction initiated by the DMA:

- **DCCM Access (0x0000_xxxx or 0xF004_xxxx):** The router forwards the transaction directly to the VeeR EL2 dedicated 64-bit AXI4 DMA slave port. Address normalization logic maps low-memory aliased addresses (0x0000_xxxx) to the physical DCCM range {16'hF004, addr[15:0]}.
- **Peripheral Access (0x4000_xxxx):** The router forwards the transaction to the dma_axi4_to_ahb_adapter, driving the AHB-Lite interconnect to access SPI, I2C, or UART registers.

2.3.6 DCCM RAM Subsystem (soc_dccm_ram)

The Data Closely Coupled Memory provides 64 KB of high-speed SRAM storage directly coupled to the processor. It operates at the full 50 MHz core frequency with zero wait states. It features dual access paths: one internal path to the VeeR EL2 execution pipeline and one external path via the core's dedicated DMA slave interface.

2.3.7 Instruction Boot ROM Subsystem (soc_ifu_boot_rom)

The Instruction Boot ROM provides 8 KB of non-volatile read-only storage mapped to base address 0x4000_0000. It is pre-loaded at simulation and synthesis time with the system firmware binary (firmware.hex). Upon power-on reset deassertion, the VeeR EL2 Instruction Fetch Unit fetches its first instruction from address 0x4000_0000.

2.3.8 SPI Master Subsystem (axi_spi_top)

The SPI controller is a Xilinx-compatible AXI Quad SPI core mapped to base address 0x4000_6000 (with an alias at 0x4000_1000). It features independent 16-entry by 32-bit Transmit (DTR) and Receive (DRR) hardware FIFOs, programmable clock polarity and phase (CPOL, CPHA), and hardware slave select generation (SSR). It connects to the AHB bus via an ahb2axi_lite_bridge.

2.3.9 I2C Master Subsystem (i2c_master_top)

The I2C master is an OpenCores-compatible Wishbone core mapped to base address 0x4000_8000 (with an alias at 0x4000_3000). It supports standard (100 kbps) and fast (400 kbps) serial speeds. It features an internal 16-bit prescaler, a byte-level transfer state machine, automatic ACK/NACK generation, and bus arbitration detection. It connects to AHB via an ahb2wishbone_bridge.

2.3.10 UART Subsystem (axi_uart_top)

The UART controller is a standard 16550-compatible serial communication core mapped to base address 0x4000_0000 (with an alias at 0x4000_2000). It includes a programmable baud rate generator, 8N1 framing, transmit holding register (THR), receive buffer register (RBR), and line status monitoring (LSR). It connects to AHB via an ahb2axi_lite_bridge.

2.3.11 Programmable Interrupt Controller (PIC) & Glue (pic_interrupt_glue)

Interrupt management is performed by the VeeR EL2 core-internal PIC (el2_pic_ctrl.sv). The PIC is mapped to address space 0xF00C_0000 and is accessed with zero wait states by the CPU LSU via an internal dedicated PICM bus. External peripheral interrupts (DMA Done, DMA Error, SPI IRQ, UART IRQ, I2C IRQ) are conditioned and synchronized by pic_interrupt_glue before driving PIC source inputs extintsrc_req[5:1].

2.4 CPU-to-Peripheral Communication Path

When firmware running on the VeeR EL2 CPU configures or queries a peripheral, the access follows a deterministic path:

- **1. Pipeline Execution:** The CPU executes a 32-bit load or store instruction (e.g., lw, sw) targeting address range 0x4000_xxxx.
- **2. Core LSU Interface:** The LSU issues a 64-bit AXI4 transaction to the u_lsu_axi2ahb bridge.
- **3. AHB Conversion:** The bridge converts the AXI transaction into an AHB-Lite transfer (HADDR, HWDATA, HWRITE).
- **4. Address Decoding:** The address decoder evaluates HADDR and asserts the appropriate peripheral select line (HSEL_SPI, HSEL_I2C, HSEL_UART, or HSEL_DMA).
- **5. Protocol Adaptation:** The target protocol bridge (AHB-to-AXI-Lite or AHB-to-Wishbone) translates AHB signaling into peripheral-native bus cycles.
- **6. Completion:** The peripheral register updates or returns data, asserting HREADY to release the CPU pipeline.

2.5 DMA-to-DCCM Communication Path (Dedicated Port Architecture)

When the DMA controller accesses DCCM SRAM, data travels over an isolated, high-performance path completely detached from the AHB-Lite bus:

- **1. DMA Request:** The DMA master drives an AXI4 read or write request targeting DCCM memory (0x0000_xxxx or 0xF004_xxxx).
- **2. Router Interception:** The dma_axi_master_router intercepts the request, detects the DCCM address range, and normalizes the upper 16 bits to 16'hF004.
- **3. Dedicated Slave Port:** The request is driven onto the VeeR EL2 dedicated 64-bit AXI4 slave port (u_veer.dma_axi_*).
- **4. Core Memory Arbitration:** The core internal DCCM arbiter grants access to the dedicated port, asserting dma_axi_aready / awready.
- **5. Single-Cycle Memory Cycle:** 64 bits of data are read from or written to DCCM SRAM in a single clock cycle, achieving maximum theoretical bandwidth.

2.6 DMA-to-SPI and DMA-to-I2C Communication Paths

To move data between DCCM and peripheral FIFOs, the DMA controller executes coordinated two-phase transactions:

- **Transmit Acceleration (TX Path):** The DMA reads a 32-bit or 64-bit payload from DCCM via the dedicated DMA slave port. Next, the DMA router directs the write to dma_axi4_to_ahb_adapter, which acquires the AHB bus and writes the data into the SPI DTR FIFO (0x4000_6068).
- **Receive Acceleration (RX Path):** The DMA adapter acquires the AHB bus to read the SPI DRR FIFO (0x4000_606C) or I2C RXR register (0x4000_800C). The received byte/word is latched into the DMA engine and written directly into DCCM via the dedicated slave port.

2.7 Interrupt Architecture Overview

The interrupt architecture utilizes an integrated, low-latency topology. Five primary hardware interrupt lines are generated across the SoC and routed into pic_interrupt_glue:

- **Source 1 (dma_done):** Asserted when a programmed DMA block transfer finishes successfully.
- **Source 2 (dma_error):** Asserted upon bus error, unaligned address access, or slave decode fault.
- **Source 3 (spi_irq):** Asserted on TX FIFO empty, RX FIFO full, or transfer overrun conditions.
- **Source 4 (uart_irq):** Asserted on RX data available (RDA) or transmitter holding register empty (THRE).
- **Source 5 (i2c_wb_inta_o):** Asserted on byte transfer complete, arbitration loss, or slave acknowledge received.

These synchronized signals enter the VeeR EL2 PIC as extintsrc_req[5:1]. The PIC prioritizes the highest pending request against the programmed threshold register (meicpct) and presents the claim to the core execution engine.

3. Block Inventory

Table 3.1 details the verified hardware modules instantiated in the VeeR EL2 Communication SoC, specifying their RTL entity names, file system locations, architectural functions, and interface connectivity.

Block Name	RTL Module Name	Source Path in Repository	Primary Architectural Function	Interface Standards	Connected Blocks
SoC Top Level	top_veer_soc	RTL/ top_veer_soc.sv	SoC top-level integration, pin muxing, bus routing, and reset/clock wiring	External Pins & Internal Buses	All Subsystems
VeeR EL2 Core	el2_veer_wrapper	RTL/Cores-VeeR-EL2-main/ design/...	RV32IMC 4-stage processor core, branch predictor, IFU/LSU pipelines	AXI4 Master (64b), AXI4 Slave (64b), PICM	Interconnect, DCCM, Boot ROM, PIC

Block Name	RTL Module Name	Source Path in Repository	Primary Architectural Function	Interface Standards	Connected Blocks
DMA Master Router	dma_axi_master_router	RTL/DMA/dma_axi_master_router.sv	Dynamic route steering: DCCM space to dedicated slave; peripheral to AHB adapter	AXI4 64-bit Master & Slave Interfaces	DMA Master, VeeR Core, AHB Adapter
DMA AHB Adapter	dma_axi4_to_ahb_adapter	RTL/DMA/dma_axi4_to_ahb_adapter.sv	Bridges 64-bit AXI4 DMA transactions to 64-bit AHB-Lite bus master transfers	AXI4 64b Slave, AHB-Lite 64b Master	DMA Router, AHB Interconnect
DMA Controller	dma_axi_wrapper / csr_dma	RTL/DMA/dma_axi_wrapper.v, csr_dma.v	Multi-mode descriptor-based DMA engine with byte counters and FSM logic	AXI4 64b Master, AHB 32b Slave (CSR)	AHB Interconnect, Router, PIC Glue
Address Decoder	address_decoder	RTL/Interconnect/address_decoder.sv	AHB address phase decoding, generating 1-hot slave selects (HSEL_*)	AHB-Lite Control Signals	AHB Bus, All AHB Slaves
LSU AXI-AHB Bridge	u_lsu_axi2ahb	RTL/Interconnect/axi2ahb.sv	Translates CPU LSU AXI4 transactions into 64-bit AHB-Lite master cycles	AXI4 64b Slave, AHB-Lite 64b Master	VeeR LSU, AHB Interconnect
DCCM SRAM Array	soc_dccm_ram	RTL/Memory/soc_dccm_ram.sv	64 KB tightly-coupled data memory array with single-cycle read/write access	Dedicated Core Memory Interface (64b)	VeeR EL2 Core Complex
Instruction Boot ROM	soc_ifu_boot_rom	RTL/Memory/soc_ifu_boot_rom.sv	8 KB instruction ROM initialized from firmware.hex containing boot code	AXI4 64b Slave / Core IFU Bus	VeeR EL2 IFU Fetch Unit
SPI Controller	axi_spi_top	RTL/SPI/axi_spi_top.v	Xilinx-compatible AXI Quad SPI master with dual 16x32 FIFOs and clock gen	AXI-Lite 32b Slave, SPI 4-Wire Pins	AHB2AXI Bridge, External SPI Pins, PIC
AHB-to-AXI-Lite SPI	ahb2axi_spi	RTL/Interconnect/ahb2axi_lite.sv	Protocol bridge adapting 64-bit AHB-Lite transfers to 32-bit AXI-Lite SPI port	AHB 64b Slave, AXI-Lite 32b Master	AHB Bus, SPI Controller

Block Name	RTL Module Name	Source Path in Repository	Primary Architectural Function	Interface Standards	Connected Blocks
I2C Master Core	i2c_master_top	RTL/I2C/ i2c_master_top.v	OpenCores I2C master with 16b prescaler, byte controller, and ACK detect	Wishbone 8/32b Slave, I2C 2-Wire Pins	AHB2WB Bridge, External I2C Pins, PIC
AHB-to-Wishbone I2C	ahb2wb_i2c	RTL/ Interconnect/ ahb2wishbone.sv	Protocol bridge adapting 64-bit AHB-Lite transfers to Wishbone I2C port	AHB 64b Slave, Wishbone Master	AHB Bus, I2C Master
UART Controller	axi_uart_top	RTL/UART/ axi_uart_top.v	16550-compatible UART core with baud divisor, line framing, and status flags	AXI-Lite 32b Slave, UART TX/RX Pins	AHB2AXI Bridge, External UART Pins, PIC
AHB-to-AXI-Lite UART	ahb2axi_uart	RTL/ Interconnect/ ahb2axi_lite.sv	Protocol bridge adapting 64-bit AHB-Lite transfers to 32-bit AXI-Lite UART port	AHB 64b Slave, AXI-Lite 32b Master	AHB Bus, UART Controller
Programmable IRQ Ctrl	el2_pic_ctrl	RTL/Cores-VeeR-EL2-main/ design/...	Core-internal PIC managing 15 priorities, gateway latches, and vector base	Internal PICM Bus (Zero-Wait LSU)	VeeR LSU, Execution Core, PIC Glue
PIC Interrupt Glue	pic_interrupt_glue	RTL/Interrupt/ pic_interrupt_glue. sv	Synchronizes, conditions, and aggregates peripheral IRQs into extintsrc_req	5x Discrete IRQ Lines, 5b Level Bus	DMA, SPI, I2C, UART, Core PIC

4. Address Map

4.1 Master System Memory Map

The VeeR EL2 Communication SoC organizes its 32-bit address space into distinct memory regions, peripheral register banks, and core-internal control spaces. Table 4.1 outlines the complete system address map extracted from the implemented RTL address decoders and verified firmware linkers.

Memory Region	Base Address	End Address	Size	Bus / Port	Access	Functional Description & Decoding
Boot ROM / Reset	0x4000_0000	0x4000_1FFF	8 KB	Core IFU / AHB	R / X	Power-on reset vector base; executes firmware.hex initialization code
UART Controller	0x4000_0000	0x4000_0BFF	3 KB	AHB-Lite Bridge	R / W	16550 UART registers (RBR/THR, IER, LCR, LSR, BAUD_DIV)
UART Alias Window	0x4000_2000	0x4000_2FFF	4 KB	AHB-Lite Bridge	R / W	Secondary peripheral alias decode window for UART controller
DMA CSR Registers	0x4000_0C00	0x4000_0FFF	1 KB	AHB-Lite Interconnect	R / W	DMA engine control, status, error registers, and descriptor 0
SPI Controller	0x4000_6000	0x4000_6FFF	4 KB	AHB-Lite Bridge	R / W	AXI Quad SPI registers (CR, SR, DTR FIFO, DRR FIFO, SSR, IER)
SPI Alias Window	0x4000_1000	0x4000_1FFF	4 KB	AHB-Lite Bridge	R / W	Secondary peripheral alias decode window for SPI controller
I2C Master Core	0x4000_8000	0x4000_8FFF	4 KB	AHB-Lite Bridge	R / W	OpenCores Wishbone I2C registers (PRER, CTR, TXR/RXR, CR/SR)
I2C Alias Window	0x4000_3000	0x4000_3FFF	4 KB	AHB-Lite Bridge	R / W	Secondary peripheral alias decode window for I2C controller

Memory Region	Base Address	End Address	Size	Bus / Port	Access	Functional Description & Decoding
ICCM SRAM Base	0xEE00_0000	0xEE00_FFFF	64 KB	Core Internal IFU	R / W / X	Instruction Closely Coupled Memory; high-speed code execution
DCCM SRAM Base	0xF004_0000	0xF004_FFFF	64 KB	Dedicated DMA / LSU	R / W	Data Closely Coupled Memory physical base; single-cycle SRAM access
DCCM Alias Window	0x0000_0000	0x0000_FFFF	64 KB	Dedicated DMA / LSU	R / W	Low-memory mirror of DCCM; normalized by router to 0xF004_xxxx
Core PIC Registers	0xF00C_0000	0xF00C_7FFF	32 KB	Internal PICM Bus	R / W	VeeR EL2 Programmable Interrupt Controller priority, mask, gateway CSRs

4.2 Peripheral Register Address Ranges

All peripheral registers reside within the 0x4000_xxxx address partition. The address decoder partitions this segment to allocate independent 4 KB or 1 KB address ranges to each IP core. Both primary and alias address windows are supported in RTL to maintain backwards compatibility across varied firmware drivers.

4.3 DCCM Address Aliasing and Normalization Logic

The physical DCCM memory is located at 0xF004_0000. However, standard RISC-V embedded compilers often position stack and static global data within the lowest 64 KB of memory (0x0000_0000 to 0x0000_FFFF) to exploit efficient absolute addressing modes.

To support this transparently without complicating software drivers, the dma_axi_master_router implements hardware address normalization logic. When the DMA issues a read or write with address bits [31:16] == 16'h0000, the router replaces the upper 16 bits with 16'hF004 prior to driving the dedicated core DMA slave interface:

[HARDWARE ROUTING NORMALIZATION] DCCM ROUTE NORMALIZATION EQUATION:

```
if ((dma_addr[31:28] == 4'h0) || (dma_addr[31:16] == 16'hF004)) begin
    target_port = DEDICATED_CORE_DMA_SLAVE;
    normalized_addr = {16'hF004, dma_addr[15:0]};
end else begin
    target_port = AHB_LITE_SYSTEM_ADAPTER;
    normalized_addr = dma_addr;
```

```
end
```

4.4 Address Decoder Implementation Details & Decoding Equations

The AHB-Lite address_decoder module decodes HADDR during the transfer address phase. The implemented combinational equations generating the slave select signals are as follows:

- **UART Select:** assign HSEL_UART = (HADDR[31:12] == 20'h40000) || (HADDR[31:12] == 20'h40002);
- **DMA CSR Select:** assign HSEL_DMA = (HADDR[31:12] == 20'h40000) && (HADDR[11:10] == 2'b11); // Range: 0x4000_0C00 - 0x4000_0FFF
- **SPI Select:** assign HSEL_SPI = (HADDR[31:12] == 20'h40006) || (HADDR[31:12] == 20'h40001);
- **I2C Select:** assign HSEL_I2C = (HADDR[31:12] == 20'h40008) || (HADDR[31:12] == 20'h40003);
- **DCCM Select:** assign HSEL_DCCM = (HADDR[31:28] == 4'h0) || (HADDR[31:16] == 16'hF004);
- **Boot ROM Select:** assign HSEL_BOOT = (HADDR[31:16] == 16'h4000) && (HADDR[15:13] == 3'b000);

5. Interface and Signal Specification

This section documents every critical bus and physical pin-level interface implemented in the VeeR EL2 Communication SoC, using the exact signal names, widths, and directions from top_veer_soc.sv and associated submodules.

5.1 64-bit AMBA AHB-Lite System Interconnect Signals

Table 5.1 defines the signals comprising the primary 64-bit AHB-Lite system interconnect fabric connecting masters (CPU LSU and DMA Adapter) to slave peripherals.

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
hclk	In	1	System bus clock (50.0 MHz synchronous timebase)	Clock Generator	All AHB Modules
hreset_n	In	1	System active-low synchronous reset	Reset Controller	All AHB Modules
haddr	Out	32	AHB byte address driven by the active bus master	Active Bus Master	Address Decoder, Slaves
hwrite	Out	1	Transfer direction: 1 = Write transfer, 0 = Read transfer	Active Bus Master	All AHB Slaves
hsize	Out	3	Transfer size: 3'b000=8b, 3'b001=16b, 3'b010=32b, 3'b011=64b	Active Bus Master	All AHB Slaves

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
hburst	Out	3	Burst type: Single (000), INCR (001), INCR4 (011), INCR8 (101)	Active Bus Master	All AHB Slaves
hprot	Out	4	Protection control: privileged, user, data, instruction access	Active Bus Master	Interconnect Arbiter
htrans	Out	2	Transfer type: IDLE (2'b00), BUSY (2'b01), NONSEQ (2'b10), SEQ (2'b11)	Active Bus Master	All AHB Slaves
hmastlock	Out	1	Locked master transfer sequence indicator (atomic bursts)	Active Bus Master	AHB Bus Arbiter
hwdata	Out	64	64-bit write data bus driven during write cycles	Active Bus Master	Selected AHB Slave
hrdata	In	64	64-bit read data bus driven by the addressed slave	Addressed Slave	Active Bus Master
hready	In/Out	1	Transfer complete handshake; high indicates slave is ready	Addressed Slave	Master & Interconnect
hresp	In	1	Transfer status: 1'b0 = OKAY response, 1'b1 = ERROR condition	Addressed Slave	Active Bus Master
hsel_spi	Out	1	Slave select for SPI controller (asserted for 0x4000_6xxx)	Address Decoder	ahb2axi_spi Bridge
hsel_i2c	Out	1	Slave select for I2C controller (asserted for 0x4000_8xxx)	Address Decoder	ahb2wb_i2c Bridge
hsel_uart	Out	1	Slave select for UART controller (asserted for 0x4000_0xxx)	Address Decoder	ahb2axi_uart Bridge

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
hsel_dma	Out	1	Slave select for DMA CSRs (asserted for 0x4000_0C00)	Address Decoder	DMA CSR Interface

5.2 DMA AXI4 Master Interface Signals (m_axi_*)

Table 5.2 defines the 64-bit AXI4 master interface signals driven by the DMA core into the dma_axi_master_router.

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
m_axi_awid	Out	4	Write address transaction ID tag	DMA Master Core	DMA Router
m_axi_awaddr	Out	32	Write address byte pointer	DMA Master Core	DMA Router
m_axi_awlen	Out	8	Write burst length (number of transfers minus 1: 0 to 255)	DMA Master Core	DMA Router
m_axi_awsize	Out	3	Write burst transfer size: 3'b010 = 32-bit, 3'b011 = 64-bit	DMA Master Core	DMA Router
m_axi_awburst	Out	2	Write burst type: 2'b00 = FIXED, 2'b01 = INCR	DMA Master Core	DMA Router
m_axi_awvalid	Out	1	Write address valid strobe	DMA Master Core	DMA Router
m_axi_awready	In	1	Write address acknowledge from slave/router	DMA Router	DMA Master Core
m_axi_wdata	Out	64	64-bit write payload data bus	DMA Master Core	DMA Router
m_axi_wstrb	Out	8	Write byte enables (1 bit per active byte lane)	DMA Master Core	DMA Router
m_axi_wlast	Out	1	Write last beat indicator for current burst	DMA Master Core	DMA Router
m_axi_wvalid	Out	1	Write data valid strobe	DMA Master Core	DMA Router

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
m_axi_wready	In	1	Write data acknowledge from slave/router	DMA Router	DMA Master Core
m_axi_bresp	In	2	Write response status: 2'b00=OKAY, 2'b10=SLVERR, 2'b11=DECERR	DMA Router	DMA Master Core
m_axi_bvalid	In	1	Write response valid strobe	DMA Router	DMA Master Core
m_axi_bready	Out	1	Write response ready acknowledgment	DMA Master Core	DMA Router
m_axi_arid	Out	4	Read address transaction ID tag	DMA Master Core	DMA Router
m_axi_araddr	Out	32	Read address byte pointer	DMA Master Core	DMA Router
m_axi_arlen	Out	8	Read burst length (number of transfers minus 1: 0 to 255)	DMA Master Core	DMA Router
m_axi_arsize	Out	3	Read burst transfer size: 3'b010 = 32-bit, 3'b011 = 64-bit	DMA Master Core	DMA Router
m_axi_arburst	Out	2	Read burst type: 2'b00 = FIXED, 2'b01 = INCR	DMA Master Core	DMA Router
m_axi_arvalid	Out	1	Read address valid strobe	DMA Master Core	DMA Router
m_axi_arready	In	1	Read address acknowledge from slave/router	DMA Router	DMA Master Core
m_axi_rdata	In	64	64-bit read payload data bus	DMA Router	DMA Master Core
m_axi_rresp	In	2	Read response status: 2'b00=OKAY, 2'b10=SLVERR, 2'b11=DECERR	DMA Router	DMA Master Core

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
m_axi_rlast	In	1	Read last beat indicator from slave	DMA Router	DMA Master Core
m_axi_rvalid	In	1	Read data valid strobe	DMA Router	DMA Master Core
m_axi_rready	Out	1	Read data ready acknowledgment	DMA Master Core	DMA Router

5.3 Dedicated VeeR EL2 DMA Slave Interface Signals (dma_axi_*)

Table 5.3 defines the 64-bit AXI4 interface signals connecting the dma_axi_master_router directly into the core DMA slave port (u_veer.dma_axi_*).

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
dma_axi_awvalid	Out	1	Write address valid targeting core DCCM	DMA Router	VeeR EL2 Core
dma_axi_awready	In	1	Core DCCM write address ready acknowledgment	VeeR EL2 Core	DMA Router
dma_axi_awaddr	Out	32	Normalized DCCM write address ({16'hF004, addr[15:0]})	DMA Router	VeeR EL2 Core
dma_axi_awsize	Out	3	Write transfer size: 3'b010=32-bit word, 3'b011=64-bit double	DMA Router	VeeR EL2 Core
dma_axi_wvalid	Out	1	Write data valid targeting core DCCM	DMA Router	VeeR EL2 Core
dma_axi_wready	In	1	Core DCCM write data ready acknowledgment	VeeR EL2 Core	DMA Router
dma_axi_wdata	Out	64	64-bit write payload data bus into DCCM	DMA Router	VeeR EL2 Core

Signal Name	Dir	Width	Description & Protocol Function	Source Block	Destination Block
dma_axi_wstrb	Out	8	Active byte write strobes for DCCM write lanes	DMA Router	VeeR EL2 Core
dma_axi_bvalid	In	1	Core DCCM write response valid	VeeR EL2 Core	DMA Router
dma_axi_bready	Out	1	Router write response ready acknowledge	DMA Router	VeeR EL2 Core
dma_axi_bresp	In	2	Write response code from core DCCM (OKAY / SLVERR)	VeeR EL2 Core	DMA Router
dma_axi_arvalid	Out	1	Read address valid targeting core DCCM	DMA Router	VeeR EL2 Core
dma_axi_arready	In	1	Core DCCM read address ready acknowledgment	VeeR EL2 Core	DMA Router
dma_axi_araddr	Out	32	Normalized DCCM read address ({16'hF004, addr[15:0]})	DMA Router	VeeR EL2 Core
dma_axi_arsize	Out	3	Read transfer size: 3'b010=32-bit word, 3'b011=64-bit double	DMA Router	VeeR EL2 Core
dma_axi_rvalid	In	1	Core DCCM read data valid strobe	VeeR EL2 Core	DMA Router
dma_axi_rready	Out	1	Router read data ready acknowledgment	DMA Router	VeeR EL2 Core
dma_axi_rdata	In	64	64-bit read payload returned directly from DCCM SRAM	VeeR EL2 Core	DMA Router
dma_axi_rresp	In	2	Read response code from core DCCM (OKAY / SLVERR)	VeeR EL2 Core	DMA Router

5.4 DCCM RAM Interface Signals

Table 5.4 specifies the physical SRAM macro interface signals connecting soc_dccm_ram to the core LSU and dedicated slave arbiter.

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
dccm_clk	In	1	Synchronous memory clock (50.0 MHz)	Clock Network	soc_dccm_ram
dccm_rst_n	In	1	Active-low synchronous memory reset	Reset Network	soc_dccm_ram
dccm_cs	In	1	SRAM chip select; activates memory bank access	Core Memory Arbiter	soc_dccm_ram
dccm_we	In	8	Byte-level write enables for 64-bit SRAM word	Core Memory Arbiter	soc_dccm_ram
dccm_addr	In	16	SRAM word address pointer (64 KB address span)	Core Memory Arbiter	soc_dccm_ram
dccm_wdata	In	64	64-bit write data bus into SRAM array	Core Memory Arbiter	soc_dccm_ram
dccm_rdata	Out	64	64-bit read data output from SRAM array (single-cycle)	soc_dccm_ram	Core Memory Arbiter
dccm_ready	Out	1	Memory ready flag; signals single-cycle cycle completion	soc_dccm_ram	VeeR Pipeline & DMA

5.5 SPI Physical and Register Interface Signals

Table 5.5 lists the physical external pins and internal interconnect signals for axi_spi_top.

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
spi_sck_o	Out	1	SPI serial shift clock (up to 25 MHz)	axi_spi_top	External SPI Bus Pin
spi_mosi_o	Out	1	Master-Out Slave-In serial data transmit line	axi_spi_top	External SPI Bus Pin

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
spi_miso_i	In	1	Master-In Slave-Out serial data receive line	External SPI Bus Pin	axi_spi_top
spi_ss_o	Out	1	Active-low slave chip select line 0	axi_spi_top	External SPI Bus Pin
spi_irq	Out	1	Interrupt request: TX FIFO empty, RX FIFO full, overrun	axi_spi_top	pic_interrupt_glue
s_axi_awaddr	In	7	AXI-Lite write address (7-bit byte offset)	ahb2axi_spi	axi_spi_top
s_axi_wdata	In	32	AXI-Lite 32-bit write data bus	ahb2axi_spi	axi_spi_top
s_axi_rdata	Out	32	AXI-Lite 32-bit read data bus (FIFO and status read)	axi_spi_top	ahb2axi_spi

5.6 I2C Physical and Register Interface Signals

Table 5.6 details the physical open-drain pads and Wishbone interface signals for i2c_master_top.

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
scl_pad_i	In	1	I2C clock input sense from external bus line	External I2C Pin	i2c_master_top
scl_pad_o	Out	1	I2C clock output driver (drives 0 to pull down SCL)	i2c_master_top	External I2C Pin Driver
scl_padoen_o	Out	1	I2C clock output enable (1 = High-Z / Float, 0 = Drive)	i2c_master_top	External I2C Pin Driver
sda_pad_i	In	1	I2C data input sense from external bus line	External I2C Pin	i2c_master_top
sda_pad_o	Out	1	I2C data output driver (drives 0 to pull down SDA)	i2c_master_top	External I2C Pin Driver
sda_padoen_o	Out	1	I2C data output enable (1 = High-Z / Float, 0 = Drive)	i2c_master_top	External I2C Pin Driver

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
i2c_wb_inta_o	Out	1	Interrupt request: Byte transferred, ACK received, AL	i2c_master_top	pic_interrupt_glue
wb_adr_i	In	3	Wishbone register byte offset (3-bit address)	ahb2wb_i2c	i2c_master_top
wb_dat_i	In	8	Wishbone 8-bit write data byte	ahb2wb_i2c	i2c_master_top
wb_dat_o	Out	8	Wishbone 8-bit read data byte	i2c_master_top	ahb2wb_i2c

5.7 UART Physical and Register Interface Signals

Table 5.7 specifies the serial communication pins and register interface signals for axi_uart_top.

Signal Name	Dir	Width	Description & Function	Source Block	Destination Block
uart_txd_o	Out	1	UART asynchronous serial transmit output	axi_uart_top	External UART Pin
uart_rxd_i	In	1	UART asynchronous serial receive input	External UART Pin	axi_uart_top
uart_irq	Out	1	Interrupt request: RX data ready (RDA), TX holding empty	axi_uart_top	pic_interrupt_glue
s_axi_awaddr	In	5	AXI-Lite write address (5-bit byte offset)	ahb2axi_uart	axi_uart_top
s_axi_wdata	In	32	AXI-Lite 32-bit write data bus (THR, IER, LCR)	ahb2axi_uart	axi_uart_top
s_axi_rdata	Out	32	AXI-Lite 32-bit read data bus (RBR, LSR, IIR)	axi_uart_top	ahb2axi_uart

5.8 PIC Interrupt Interface Signals

Table 5.8 specifies the interrupt request inputs and synchronization signals connecting pic_interrupt_glue to el2_pic_ctrl.sv.

Signal Name	Dir	Width	Description & Mapping	Source Block	Destination Block
extintsrc_req[1]	In	1	Source 1: DMA transfer complete interrupt (dma_done)	pic_interrupt_glue	VeeR EL2 PIC
extintsrc_req[2]	In	1	Source 2: DMA bus error / fault interrupt (dma_error)	pic_interrupt_glue	VeeR EL2 PIC
extintsrc_req[3]	In	1	Source 3: SPI peripheral interrupt (spi_irq)	pic_interrupt_glue	VeeR EL2 PIC
extintsrc_req[4]	In	1	Source 4: UART peripheral interrupt (uart_irq)	pic_interrupt_glue	VeeR EL2 PIC
extintsrc_req[5]	In	1	Source 5: I2C peripheral interrupt (i2c_wb_inta_o)	pic_interrupt_glue	VeeR EL2 PIC
picm_maddr	In	32	Core LSU direct PICM register address (0xF00C_xxxx)	VeeR EL2 LSU	VeeR EL2 PIC
picm_wdata	In	32	Core LSU direct PICM write data	VeeR EL2 LSU	VeeR EL2 PIC
picm_rdata	Out	32	Core LSU direct PICM read data (zero-wait)	VeeR EL2 PIC	VeeR EL2 LSU

5.9 Clock and Reset Signals

Table 5.9 summarizes the global timing and reset control signals across the SoC hierarchy.

Signal Name	Dir	Width	Description & Timing Characteristic	Source Block	Destination Block
clk	In	1	Primary system clock: 50.0 MHz, 50% duty cycle	External Oscillator	All Flip-Flops & Memories
rst_n	In	1	Global master asynchronous active-low reset	External Reset Pad	Reset Synchronizer Tree
hreset_n	Internal	1	Synchronous active-low system bus reset	Reset Synchronizer	AHB Interconnect & Bridges

Signal Name	Dir	Width	Description & Timing Characteristic	Source Block	Destination Block
core_rst_n	Internal	1	Synchronous active-low core pipeline reset	Reset Synchronizer	VeeR EL2 Core Complex

6. Register Specification

This section documents the exact register bit-fields, offsets, access permissions, reset values, and behavioral descriptions extracted directly from the verified RTL sources and firmware driver headers.

6.1 DMA Controller Registers (Base: 0x4000_0C00)

The DMA subsystem control and status registers occupy a 1 KB window at base address 0x4000_0C00, implemented in csr_dma.v.

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x00	DMA_CTRL	32-bit	R / W	0x0000_0000	Master DMA Channel Control (Start, Abort, Mode, Interrupt Enable)
0x08	DMA_STATUS	32-bit	RO / W1C	0x0000_0000	Channel Status (Busy, Done, Error, FSM State, Remaining Bytes)
0x10	DMA_ERROR_ADDR	32-bit	RO	0x0000_0000	Bus Error Address Latch (Captures faulting address on AXI/AHB error)
0x18	DMA_ERROR_STATS	32-bit	RO / W1C	0x0000_0000	Detailed Error Flags (Bus Error, Address Misalignment, Size Violation)
0x20	DMA_DESC0_SRC	32-bit	R / W	0x0000_0000	Channel 0 Source Address Pointer (DCCM or Peripheral FIFO address)
0x30	DMA_DESC0_DST	32-bit	R / W	0x0000_0000	Channel 0 Destination Address Pointer (DCCM or Peripheral FIFO address)

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x40	DMA_DESC0_BYTES	32-bit	R / W	0x0000_0000	Transfer Length in Bytes (1 to 65,536 bytes; 0 = Inactive)
0x50	DMA_DESC0_CFG	32-bit	R / W	0x0000_0000	Transfer Configuration (Burst length, source/dest increment, widths)

DMA_CTRL Register (Offset: 0x00)

Bits	Field Name	Access	Reset	Description & Operational Behavior
0	START	WO	0	Channel Start: Write 1 to launch programmed transfer. Self-clearing.
1	ABORT	WO	0	Transfer Abort: Write 1 to force immediate FSM transition to IDLE.
2	IE	RW	0	Interrupt Enable: 1 = Assert dma_done / dma_error IRQ to PIC.
4:3	MODE[1:0]	RW	00	Transfer Mode: 2'b00 = M2M, 2'b01 = M2P (TX), 2'b10 = P2M (RX).
7:5	BURST_SZ[2:0]	RW	000	Max Burst Beat Size: 000=Single beat, 001=4 beats, 010=8 beats, 011=16 beats.
8	CH_EN	RW	0	Channel Enable: 1 = Channel active, 0 = Disabled.
31:9	RESERVED	RO	0	Reserved bits. Always write 0.

DMA_STATUS Register (Offset: 0x08)

Bits	Field Name	Access	Reset	Description & Operational Behavior
0	BUSY	RO	0	DMA Busy Flag: 1 = Engine is actively executing transfers.
1	DONE	W1C	0	Transfer Done Flag: Set high on completion. Write 1 to clear.
2	ERROR	W1C	0	Error Flag: Set high on bus error or alignment violation. Write 1 to clear.
6:3	FSM_STATE[3:0]	RO	0000	Current FSM State: 0=IDLE, 1=FETCH, 2=READ, 3=WRITE, 4=DONE, 5=ERR.
31:16	BYTES_LEFT[15:0]	RO	0x0000	Remaining Transfer Bytes: Real-time decrementing byte counter.

6.2 SPI Controller Registers (Base: 0x4000_6000)

The SPI subsystem registers implement the Xilinx-compatible AXI Quad SPI architecture. Base address is 0x4000_6000 (alias 0x4000_1000).

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x1C	GIER	32-bit	R / W	0x0000_0000	Global Interrupt Enable: Bit 31 enables IP interrupt assertion.
0x20	ISR	32-bit	RO / W1C	0x0000_0000	Interrupt Status Register: Displays pending SPI events.
0x28	IER	32-bit	R / W	0x0000_0000	Interrupt Enable Register: Enables specific event flags to generate IRQ.

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x40	SRR	32-bit	WO	0x0000_0000	Software Reset Register: Write 0x0000_000A to soft-reset SPI core.
0x60	CR	32-bit	R / W	0x0000_0180	SPI Control Register: Master enable, CPOL/CPHA, FIFO resets, manual SS.
0x64	SR	32-bit	RO	0x0000_0005	SPI Status Register: FIFO empty/full flags, transfer progress.
0x68	DTR	32-bit	WO	0x0000_0000	Data Transmit Register: Top of 16-deep 32-bit Transmit FIFO.
0x6C	DRR	32-bit	RO	0x0000_0000	Data Receive Register: Top of 16-deep 32-bit Receive FIFO.
0x70	SSR	32-bit	R / W	0xFFFF_FFFF	Slave Select Register: Active-low bitmask (Bit 0 asserts spi_ss_o[0]).

SPI Control Register (CR, Offset: 0x60)

Bits	Field Name	Access	Reset	Description & Operational Behavior
0	LOOP	RW	0	Loopback Mode: 1 = Internal loopback enabled, 0 = Normal pin operation.
1	SPE	RW	0	SPI System Enable: 1 = Core operational, 0 = Core disabled.
2	MASTER	RW	0	Master Mode Select: 1 = Master mode (always 1 for this SoC).

Bits	Field Name	Access	Reset	Description & Operational Behavior
3	CPOL	RW	0	Clock Polarity: 0 = SCK low when idle, 1 = SCK high when idle.
4	CPHA	RW	0	Clock Phase: 0 = Data captured on first edge, 1 = Second edge.
5	TX_RST	WO	0	TX FIFO Reset: Write 1 to flush Transmit FIFO.
6	RX_RST	WO	0	RX FIFO Reset: Write 1 to flush Receive FIFO.
7	MANUAL_SS	RW	1	Manual Slave Select Assertion: 1 = Driven by SSR, 0 = Automatic.

6.3 I2C Master Controller Registers (Base: 0x4000_8000)

The I2C master controller registers implement the OpenCores Wishbone architecture at base address 0x4000_8000 (alias 0x4000_3000).

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x00	PRER_LO	8-bit	R / W	0xFF	Clock Prescaler Low Byte: SCL frequency divider low byte.
0x04	PRER_HI	8-bit	R / W	0xFF	Clock Prescaler High Byte: SCL frequency divider high byte.
0x08	CTR	8-bit	R / W	0x00	Control Register: Bit 7=EN (Core Enable), Bit 6=IEN (Interrupt Enable).
0x0C	TXR / RXR	8-bit	WO / RO	0x00	Transmit Register (Write) / Receive Register (Read).
0x10	CR / SR	8-bit	WO / RO	0x00	Command Register (Write) / Status Register (Read).

I2C Command & Status Registers (Offset: 0x10)

Bits	Field Name	Access	Description & Operational Behavior
CR[7]	STA	WO	Generate repeated START condition on bus.
CR[6]	STO	WO	Generate STOP condition on bus.
CR[5]	RD	WO	Read byte from I2C bus slave.
CR[4]	WR	WO	Write byte in TXR to I2C bus slave.
CR[3]	ACK	WO	Acknowledge value to drive during read cycle: 0=ACK, 1=NACK.
CR[0]	IACK	WO	Interrupt Acknowledge: Write 1 to clear pending interrupt flag.
SR[7]	RXACK	RO	Received Acknowledge from slave: 0 = ACK received, 1 = NACK received.
SR[6]	BUSY	RO	Bus Busy: 1 = I2C bus is actively held; set by START, cleared by STOP.
SR[5]	AL	RO	Arbitration Lost: 1 = Core lost multi-master bus arbitration.
SR[1]	TIP	RO	Transfer In Progress: 1 = Byte transmission or reception active.
SR[0]	IF	RO	Interrupt Flag: 1 = Byte transferred or arbitration lost.

6.4 UART Controller Registers (Base: 0x4000_0000)

The UART controller implements standard 16550 registers at base address 0x4000_0000 (alias 0x4000_2000).

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x00	RBR / THR	8-bit	RO / WO	0x00	Receive Buffer Register (RO) / Transmitter Holding Register (WO).

Offset	Register Name	Width	Access	Reset Value	Functional Description
0x04	IER	8-bit	R / W	0x00	Interrupt Enable Register: Bit 0=ERBFI (RX ready), Bit 1=ETBEI (TX empty).
0x08	BAUD_DIVISOR	16-bit	R / W	0x01B2	Baud Rate Divisor Latch: Divisor = 50,000,000 / (16 * Baud Rate).
0x0C	LCR	8-bit	R / W	0x03	Line Control Register: 8 data bits, 1 stop bit, no parity (8N1).
0x14	LSR	8-bit	RO	0x60	Line Status Register: Bit 0=DR (Data Ready), Bit 5=THRE, Bit 6=TEMT.

6.5 VeeR EL2 PIC & RISC-V Core CSRs

The VeeR EL2 Programmable Interrupt Controller registers are memory-mapped to 0xF00C_0000. Core-internal CSRs are accessed via standard RISC-V csrr/csrw instructions.

Address / CSR	Register Name	Access	Description & Interrupt Control Function
0xF00C_0004	meipl[1]	R / W	Source 1 Priority Level: DMA Done interrupt priority (0 to 15).
0xF00C_0008	meipl[2]	R / W	Source 2 Priority Level: DMA Bus Error interrupt priority (0 to 15).
0xF00C_000C	meipl[3]	R / W	Source 3 Priority Level: SPI peripheral interrupt priority (0 to 15).
0xF00C_0010	meipl[4]	R / W	Source 4 Priority Level: UART peripheral interrupt priority (0 to 15).
0xF00C_0014	meipl[5]	R / W	Source 5 Priority Level: I2C peripheral interrupt priority (0 to 15).
0xF00C_1004..14	meip[1..5]	RO / W1C	Gateway Interrupt Pending Registers: Displays active pending peripheral requests.

Address / CSR	Register Name	Access	Description & Interrupt Control Function
0xF00C_3000	meicpct	RO	Claim Pending Context: Returns ID and priority of highest pending interrupt.
0xF00C_3004	meipt	R / W	Claim Priority Threshold: Interrupts with priority <= threshold are masked.
CSR 0xBC8	meivt	R / W	Machine External Interrupt Vector Table Base Address.
CSR 0x300	mstatus	R / W	Machine Status: Bit 3 (MIE) provides global interrupt enable.
CSR 0x304	mie	R / W	Machine Interrupt Enable: Bit 11 (MEIE) enables external PIC interrupts.
CSR 0x344	mip	RO	Machine Interrupt Pending: Bit 11 (MEIP) reflects PIC trigger to core.

7. Detailed Control Path

7.1 CPU Control Path Architecture

The CPU control path encompasses all instructions executed by the VeeR EL2 core to program peripheral configurations, initiate DMA channels, manage hardware FIFOs, and service interrupts. Figure 7.1 details the bus pathways and address translation mechanics utilized by the CPU.

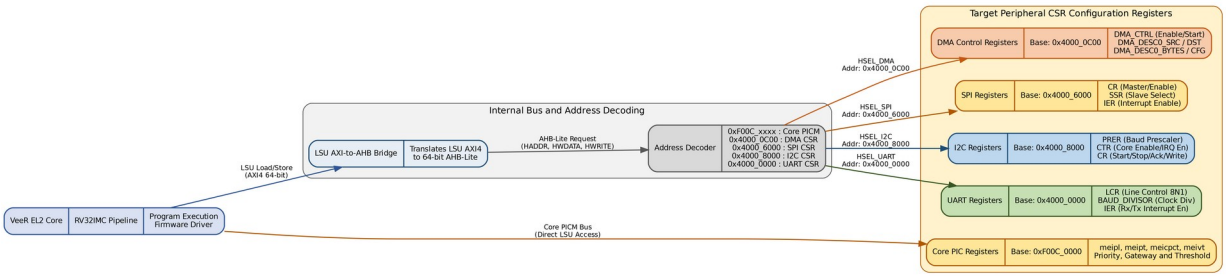


Figure 7.1: *CPU Control Path Diagram showing VeeR EL2 LSU instruction execution, AXI-to-AHB bridge, Address Decoder, and Peripheral CSR Mapping.*

7.2 Step-by-Step Firmware Programming Sequences

7.2.1 CPU Initialization Sequence

- **1. Core Setup:** Set exception vector base, initialize stack pointer (sp = 0x0000_2000), and configure Machine Trap Vector (mtvec).
- **2. PIC Setup:** Write meipt = 0 (allow all priorities > 0), set meipl[1..5] to assigned levels (1..15), and set meivt to the vector table base.

- **3. CSR Unmask:** Write mstatus.MIE = 1 and mie.MEIE = 1 to unmask external interrupts in the RISC-V core pipeline.

7.2.2 DMA Transfer Configuration & Descriptor Setup

- **Step 1:** Firmware writes DMA_DESC0_SRC = 0xF004_1000 (Source DCCM buffer pointer).
- **Step 2:** Firmware writes DMA_DESC0_DST = 0x4000_6068 (Destination SPI DTR FIFO address).
- **Step 3:** Firmware writes DMA_DESC0_BYTES = 64 (Payload length: 64 bytes).
- **Step 4:** Firmware writes DMA_DESC0_CFG = 0x0000_0121 (SRC_INC=1, DST_INC=0, BURST=4 beats).
- **Step 5:** Firmware writes DMA_CTRL = 0x0000_0105 (CH_EN=1, IE=1, MODE=M2P, START=1).

7.2.3 SPI Peripheral Setup & Activation

- **1. Soft Reset:** Write SPI SRR (0x40) = 0x0A (Software reset; flushes FIFOs and resets shift registers).
- **2. Mode Config:** Write SPI CR (0x60) = 0x0186 (SPE=1, Master=1, Manual SS=1, CPOL=0, CPHA=0).
- **3. Slave Select:** Write SPI SSR (0x70) = 0xFFFFFFFE (Asserts chip select 0 low to target slave).

7.2.4 I2C Peripheral Setup & Command Execution

- **1. Clock Prescale:** Write PRER_LO = 0x63, PRER_HI = 0x00 (Prescaler = 99 for 100 kHz SCL from 50 MHz clock).
- **2. Core Enable:** Write CTR = 0x80 (Core Enable bit EN = 1).
- **3. START & Addr:** Write TXR = {Slave_Addr, 1'b0} (Slave write address), then write CR = 0x90 (STA + WR).
- **4. Handshake:** Poll SR[1] (TIP) until transmission finishes, verify SR[7] (RXACK == 0).

7.3 Interrupt Servicing & Completion Sequence

When a transfer concludes, hardware and software interact via a deterministic handshake:

- **1. Hardware Trigger:** DMA asserts dma_done; pic_interrupt_glue pulses extintsrc_req[1].
- **2. PIC Arbitration:** VeeR PIC latches meip[1], evaluates priority > meipt, and drives core interrupt line.
- **3. Pipeline Trap:** Core completes current instruction, pushes PC to mepc, sets mcause = 0x8000000B, and vectors to ISR.
- **4. Claim:** ISR reads meicpct to identify Source 1 (DMA Done).
- **5. Acknowledge:** ISR writes 1 to DMA_STATUS.DONE (clearing flag) and writes 1 to meip[1] (clearing PIC gateway).
- **6. Return:** ISR executes mret instruction; core resumes baseline execution.

8. Detailed Data Path

8.1 DMA Data Path Architecture

The SoC data path has been architected to achieve maximum transfer throughput between memory and peripherals while eliminating bus bottlenecks. Figure 8.1 depicts the DMA data routing mechanism, showing the separation between dedicated DCCM accesses and peripheral AHB-Lite transfers.

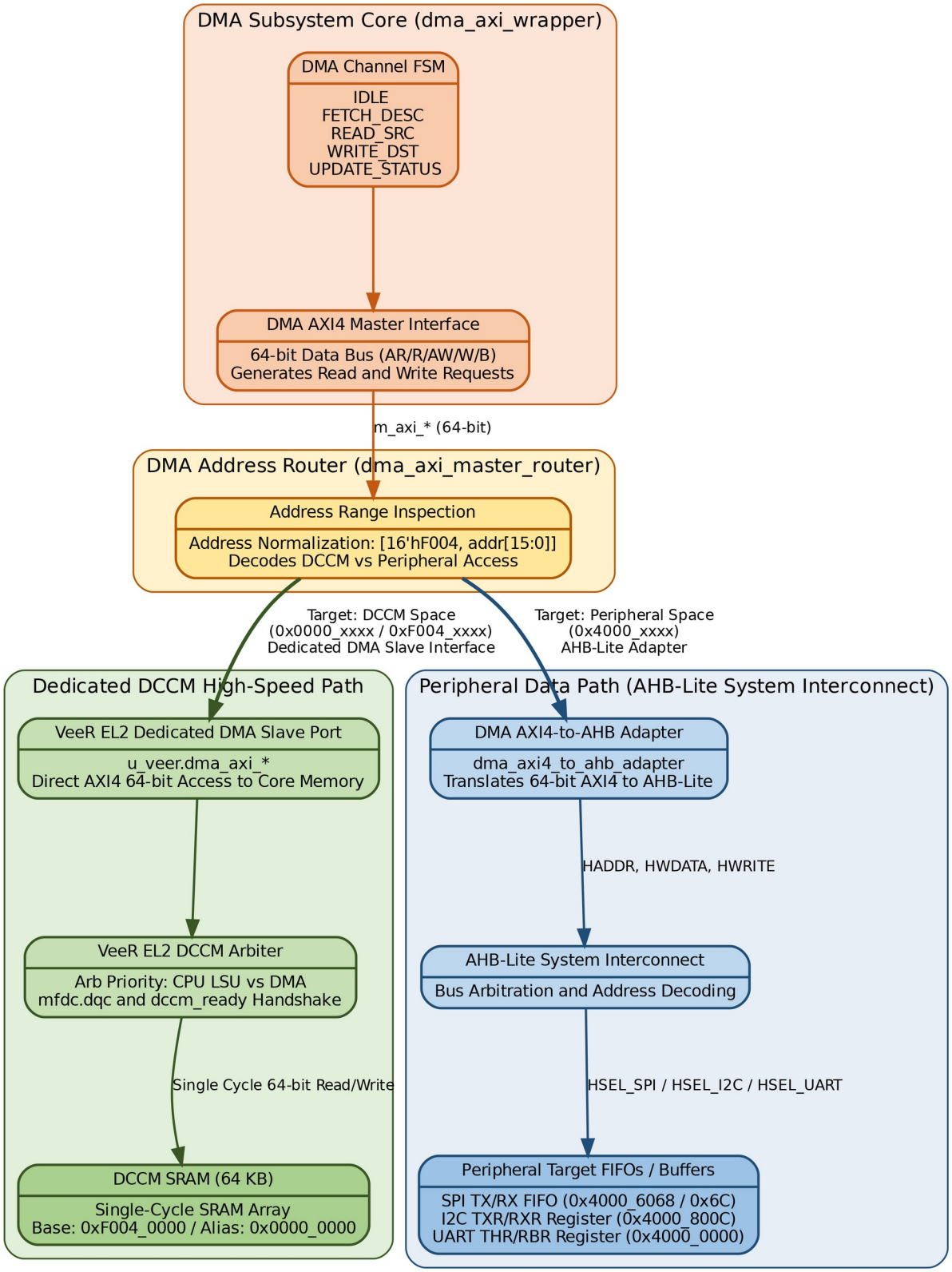


Figure 8.1: *DMA Data Path Diagram showing DMA Engine, Address Router, Dedicated 64-bit Core DMA Slave Port, DCCM SRAM, and AHB-Lite Interconnect.*

8.2 Transmit Data Path: DCCM to SPI/I2C

During memory-to-peripheral (TX) acceleration, data flows along the following stages:

- **Phase 1 (DCCM Read):** The DMA master drives m_axi_araddr = 0xF004_1000. The router forwards this to the dedicated slave port (dma_axi_araddr). The core DCCM arbiter grants access, and DCCM returns 64 bits of data in 1 clock cycle.
- **Phase 2 (Internal Slicing):** The 64-bit data word is latched into the DMA internal holding register. The byte-packing engine splits the double-word into 32-bit or 8-bit slices depending on target peripheral width.
- **Phase 3 (AHB Write):** The DMA router directs m_axi_awaddr = 0x4000_6068 (SPI DTR) to dma_axi4_to_ahb_adapter. The adapter initiates an AHB write (HADDR=0x4000_6068, HWDATA=data_word).
- **Phase 4 (FIFO Push):** The ahb2axi_spi bridge converts AHB to AXI-Lite and pushes the word into the 16-deep SPI Transmit FIFO.
- **Phase 5 (Serialization):** The SPI shift register pops the FIFO and shifts bits serially out of spi_mosi_o on consecutive SCK clock edges.

8.3 Receive Data Path: SPI/I2C to DCCM

During peripheral-to-memory (RX) acceleration, the process operates in reverse:

- **Phase 1 (Deserialization):** Serial bits arriving on spi_miso_i or sda_pad_i are deserialized into internal byte/word registers. For SPI, words accumulate in the 16-deep DRR FIFO.
- **Phase 2 (Peripheral Read):** The DMA adapter acquires the AHB bus and executes an AHB read to SPI DRR (0x4000_606C) or I2C RXR (0x4000_800C).
- **Phase 3 (Packing):** The incoming bytes are assembled into a 64-bit quad-word in the DMA engine holding register.
- **Phase 4 (DCCM Single-Cycle Write):** The DMA router steers m_axi_awaddr = 0xF004_2000 into the dedicated core DMA slave port. The write completes into DCCM SRAM in a single clock cycle.

8.4 Bus Widths, Lane Steering & Transfer Latencies

Table 8.1 details the bus parameters, lane steering mechanics, and cycle latencies across each stage of the data path.

Data Path Segment	Bus Protocol	Native Data Width	Transfer Sizing	Typical Latency (Cycles)
Core LSU to DCCM	Direct Core Bus	64-bit	8-bit, 16-bit, 32-bit, 64-bit	1 cycle (Zero wait states)
DMA to Dedicated Slave Port	AMBA AXI4	64-bit	32-bit word, 64-bit double	1 cycle address + 1 cycle data
DMA to AHB System Interconnect	AMBA AHB-Lite	64-bit	32-bit word, 64-bit burst	2 cycles per single beat
AHB to SPI Controller	AXI-Lite Bridge	32-bit	32-bit word (Lane [31:0])	3 cycles (Bridge + FIFO push)
AHB to I2C Master	Wishbone Bridge	8-bit	8-bit byte (Lane [7:0])	4 cycles (Bridge + WB handshake)
AHB to UART Controller	AXI-Lite Bridge	8/32-bit	8-bit byte (Lane [7:0])	3 cycles (Bridge + Reg write)

9. DMA Architecture and Operation

9.1 DMA Engine Architecture & FSM Pipeline

The DMA subsystem core (dma_axi_wrapper) is a high-performance transfer engine governed by a robust finite state machine. The FSM states include:

- **IDLE (4'b0000):** Channel idle; waiting for DMA_CTRL.START assertion.
- **FETCH_DESC (4'b0001):** Latches programmed descriptors (SRC, DST, BYTES, CFG) and calculates burst boundaries.
- **READ_SRC (4'b0010):** Drives AXI read address channel (AR) and collects read beats on R data channel.
- **WRITE_DST (4'b0011):** Drives AXI write address channel (AW) and streams beats on W data channel.
- **BURST_UPDATE (4'b0100):** Decrements remaining byte counter. Loops back to READ_SRC if BYTES > 0.
- **DONE (4'b0101):** Asserts dma_done interrupt line, updates DMA_STATUS.DONE, and returns to IDLE.
- **ERROR (4'b0110):** Latches fault address into DMA_ERROR_ADDR, asserts dma_error IRQ, and halts.

9.2 Transfer Modes & Descriptor Engine

The DMA controller supports three primary operational modes:

- **Memory-to-Memory (M2M):** Moves data blocks between two DCCM regions. Both source and destination accesses use the dedicated 64-bit core DMA slave port, achieving maximum data throughput (up to 400 MB/s at 50 MHz).
- **Memory-to-Peripheral (M2P):** Reads source data from DCCM via the dedicated slave port and writes into peripheral FIFOs (SPI DTR / UART THR / I2C TXR) across the AHB interconnect.
- **Peripheral-to-Memory (P2M):** Reads incoming peripheral data across AHB and deposits payloads into DCCM via the dedicated slave port.

9.3 Dedicated VeeR EL2 DMA Slave Interface Integration

In strict adherence to the Western Digital VeeR EL2 Programmer's Reference Manual, the core provides an external 64-bit AXI4 slave port dedicated exclusively to DCCM access. Integrating this port directly with the DMA router yields three critical advantages:

- Eliminates AHB bus contention between DMA DCCM accesses and CPU peripheral accesses.
- Guarantees deterministic, single-cycle SRAM access timing.
- Preserves full 64-bit data path bandwidth directly into the memory core.

9.4 32-bit and 64-bit Alignment Rules & Access Sizing

The VeeR EL2 DCCM memory array is organized as 64-bit words with byte-write enables. To prevent memory corruption and ECC faults, the DMA enforces strict alignment rules:

- **Double-Word (64-bit):** 64-bit transfers must be 8-byte aligned (addr[2:0] == 3'b000). Strobes wstrb[7:0] are asserted simultaneously.
- **Word (32-bit):** 32-bit transfers must be 4-byte aligned (addr[1:0] == 2'b00). For lower-word access (addr[2]==0), wstrb[3:0] is asserted; for upper-word (addr[2]==1), wstrb[7:4] is asserted.
- **Error Handling:** If an unaligned address is programmed, the DMA halts immediately and raises the ADDR_UNALIGNED error flag in DMA_ERROR_STATS.

9.5 CPU vs DMA DCCM Arbitration & mfdc.dqc Handshake

When the CPU LSU and the dedicated DMA slave port simultaneously request DCCM access, the VeeR EL2 core internal arbiter resolves contention:

- **Default Arbitration:** By default, the CPU LSU is granted priority to prevent pipeline stalls during time-critical loops.

- **DMA Queue Control (DQC):** The RISC-V Machine Feature Disable Control CSR (mfdc.dqc, bit 18) controls DMA queue throttling, ensuring that burst DMA accesses do not starve the processor.
- **Throttle Handshake:** The memory controller asserts dccm_ready to throttle the DMA when the CPU claims a back-to-back memory cycle.

10. Peripheral Specifications

10.1 Serial Peripheral Interface (SPI)

Figure 10.1 illustrates the complete hardware data path for DMA-accelerated SPI transfers.



Figure 10.1: *DMA-Accelerated SPI Transfer Architecture showing DCCM Buffer, DMA Engine, AHB Interconnect, AXI-Lite Bridge, Dual 16-deep FIFOs, and Physical SPI Pins.*

10.1.1 Overview & Functional Features

The SPI controller (axi_spi_top) is an AMBA AXI Quad SPI master IP core fully compatible with industry-standard serial flash, sensor, and display devices. Features include:

- Dual 16-entry deep by 32-bit wide Transmit (DTR) and Receive (DRR) FIFOs.
- Full duplex master operation with programmable baud rate up to clk/2 (25 MHz).
- Configurable clock polarity (CPOL) and phase (CPHA) supporting Modes 0, 1, 2, and 3.
- Manual and automatic chip select assertion (spi_ss_o[0]).
- Independent interrupt generation for TX FIFO empty, RX FIFO full, and overrun events.

10.1.2 Clocking and Protocol Modes

Table 10.1 outlines the four SPI operational modes supported by the core.

Mode	CPOL	CPHA	SCK Idle State	Data Capture Edge	Data Shift Edge
Mode 0	0	0	Low (0)	Rising Edge (First)	Falling Edge (Second)
Mode 1	0	1	Low (0)	Falling Edge (Second)	Rising Edge (First)
Mode 2	1	0	High (1)	Falling Edge (First)	Rising Edge (Second)
Mode 3	1	1	High (1)	Rising Edge (Second)	Falling Edge (First)

10.2 Inter-Integrated Circuit (I²C) Master Interface

Figure 10.2 depicts the DMA-accelerated I2C transmission and reception architecture.



Figure 10.2: *DMA-Accelerated I2C Transfer Architecture showing DCCM Buffer, DMA Engine, AHB Interconnect, Wishbone Bridge, I2C Master Core, and Physical Open-Drain Bus Pins.*

10.2.1 Overview & Functional Features

The I2C master controller (i2c_master_top) provides an OpenCores-compatible two-wire serial interface supporting multi-byte master transfers. Features include:

- Support for Standard mode (100 kbps) and Fast mode (400 kbps) operation.
- Programmable 16-bit clock prescaler derived from the 50 MHz system clock.
- Automated generation of START, Repeated START, and STOP conditions.
- Hardware acknowledge detection (ACK/NACK) and multi-master arbitration loss detection.
- DMA handshake integration enabling continuous burst streaming.

10.2.2 Clock Prescaler Calculation

The I2C serial clock (SCL) frequency is configured via prescale registers PRER_LO and PRER_HI according to the formula:

> **[I2C SCL PRESCALER CALCULATION]** I2C PRESCALER FORMULA:
> PRER = (System_Clock / (5 * SCL_Frequency)) - 1
>
> Example for 100 kHz Standard Mode (System Clock = 50 MHz):
> PRER = (50,000,000 / (5 * 100,000)) - 1 = 100 - 1 = 99 (0x0063)
> Result: Write PRER_LO = 0x63, PRER_HI = 0x00.
>
> Example for 400 kHz Fast Mode (System Clock = 50 MHz):
> PRER = (50,000,000 / (5 * 400,000)) - 1 = 25 - 1 = 24 (0x0018)
> Result: Write PRER_LO = 0x18, PRER_HI = 0x00.

10.3 Universal Asynchronous Receiver Transmitter (UART)

10.3.1 Overview & Functional Features

The UART controller (axi_uart_top) implements an industry-standard 16550 architecture. Features include:

- Standard 8N1 serial frame format (8 data bits, no parity, 1 stop bit).
- Programmable 16-bit baud rate divisor matching standard rates (115200, 57600, 38400, 9600 bps).
- Holding registers for transmit (THR) and receive (RBR) with line status flags (LSR.THRE, LSR.DR).
- Dedicated interrupt output (uart_irq) routed directly to PIC Source 4.

10.3.2 Baud Rate Generator Formula

The UART baud rate is calculated using the 16-bit BAUD_DIVISOR register:

> **[UART BAUD RATE CALCULATION]** UART BAUD DIVISOR FORMULA:
> BAUD_DIVISOR = System_Clock / (16 * Baud_Rate)
>
> Example for 115,200 bps (System Clock = 50 MHz):
> BAUD_DIVISOR = 50,000,000 / (16 * 115,200) = 50,000,000 / 1,843,200 ≈ 27.126 -> Divisor = 27 (0x001B)
> Actual Baud: 115,740 bps (0.47% error, well within standard 2% UART tolerance).
>
> Example for 9,600 bps (System Clock = 50 MHz):
> BAUD_DIVISOR = 50,000,000 / (16 * 9,600) = 50,000,000 / 153,600 ≈ 325.52 -> Divisor = 326 (0x0146).

11. Interrupt Specification

11.1 Interrupt Architecture Overview

The SoC implements a tightly-coupled, prioritized interrupt architecture managed directly by the VeeR EL2 core-internal Programmable Interrupt Controller (PIC). Figure 11.1 illustrates the interrupt aggregation, synchronization, priority filtering, and CPU dispatch flow.



Figure 11.1: *Interrupt Routing Architecture showing Peripheral IRQ Sources, Glue Synchronization Logic, VeeR EL2 Internal PIC, Priority Gateway, and Core CSR Dispatch.*

11.2 PIC Source Allocation & Priority Levels

The internal PIC manages five external interrupt sources (extintsrc_req[5:1]). Table 11.1 details the hardware source assignments, trigger mechanisms, and programmed priorities.

PIC Source ID	Hardware IRQ Signal	Generating Subsystem	Trigger Characteristic	Default Priority	Interrupt Cause / Description
Source 1	dma_done	DMA Controller	Active-High Level	Priority 14 (High)	DMA transfer finished successfully; block bytes moved.
Source 2	dma_error	DMA Controller	Active-High Level	Priority 15 (Max)	DMA bus error, slave response error, or unaligned access.
Source 3	spi_irq	AXI Quad SPI	Active-High Level	Priority 10 (Med)	SPI TX FIFO empty, RX FIFO full, or transfer overrun.
Source 4	uart_irq	16550 UART	Active-High Level	Priority 8 (Med)	UART Receiver Data Available (RDA) or THR Empty (THRE).
Source 5	i2c_wb_inta_o	Wishbone I2C Master	Active-High Level	Priority 9 (Med)	I2C byte transfer finished, ACK received, or arbitration lost.

11.3 Interrupt Conditioning & Synchronization (pic_interrupt_glue)

Peripheral interrupt lines originating in distinct modules pass through pic_interrupt_glue before entering the core PIC. The glue module performs:

- **Metastability Filtering:** Double-flop synchronization into the 50 MHz core clock domain to eliminate metastability on asynchronous peripheral signals.

- **Gateway Holding:** Pulse-to-level conditioning, holding request lines active until explicitly cleared by software ISR acknowledgements.
- **Bus Bundling:** Array bundling of discrete wires into extintsrc_req[5:1].

11.4 RISC-V Core Interrupt Processing Sequence

The VeeR EL2 core processes external interrupts via standard RISC-V machine-mode CSR mechanisms:

- **Enable Condition:** External interrupts are enabled when mstatus.MIE = 1 and mie.MEIE = 1.
- **Pending State:** When the PIC asserts the external interrupt line, mip.MEIP reflects the pending condition.
- **Trap Entry:** The core pipeline completes the current instruction, stores the return address in mepc, writes mcause = 0x8000000B (Machine External Interrupt), and branches to the base address stored in mtvec or meivt.
- **Claim & Return:** The ISR reads meicpct to claim the highest priority interrupt, executes the corresponding driver handler, clears the peripheral status flag, and executes mret to resume the interrupted thread.

12. Clock and Reset Specification

12.1 Clock Domain Architecture

To maximize timing closure reliability, simplify verification, and avoid complex asynchronous clock domain crossing (CDC) synchronizers, the VeeR EL2 Communication SoC operates on a single synchronous system clock domain:

- **System Clock:** The primary clock input (clk) operates at a nominal frequency of 50.0 MHz (20.0 ns period, 50% duty cycle).
- **Unified Distribution:** The 50 MHz clock is distributed synchronously to the VeeR EL2 processor core, the 64-bit AHB-Lite interconnect, the DMA controller, the DCCM and Boot ROM SRAM arrays, and all peripheral IP cores.
- **Synchronous Prescaling:** Peripheral serial baud rates (such as SPI SCK, I2C SCL, and UART baud rates) are generated entirely via internal synchronous digital clock divider prescalers, ensuring zero asynchronous boundary crossings inside the silicon fabric.

12.2 Reset Architecture & Reset Tree

The reset system features a robust, glitch-free active-low architecture:

- **Master Reset:** The top-level rst_n pin provides an active-low master reset.
- **Synchronous Deassertion:** An on-chip reset synchronizer macro latches rst_n asynchronously on assertion and releases it synchronously on the rising edge of clk, preventing reset recovery/removal timing violations across sequential flip-flops.
- **Reset Tree Distribution:** Synchronized reset signals (hreset_n, core_rst_n) drive balanced clock-tree reset branches feeding the CPU, interconnect, DMA engine, and peripherals.

12.3 Power-On Initialization & Boot Sequence

Upon power-on reset release, the SoC executes a deterministic hardware boot sequence:

- **T0 (Reset Release):** Hardware initializes all internal registers, clears FIFOs, and resets FSMs to IDLE within 5 clock cycles of rst_n deassertion.
- **T1 (Fetch Reset Vector):** The VeeR EL2 Instruction Fetch Unit drives address 0x4000_0000 on its instruction bus, targeting the 8 KB Boot ROM.
- **T2 (Boot Instruction Execution):** soc_ifu_boot_rom delivers the first instruction word from firmware.hex.
- **T3 (Runtime Initialization):** Firmware initializes register state, sets up the C runtime environment (stack pointer, .bss clearing, .data copying into DCCM), configures the PIC and peripherals, and launches the application payload.

13. Verification Architecture

13.1 Verification Strategy & UVM Testbench Environment

The SoC has been rigorously verified using an industry-standard Universal Verification Methodology (UVM 1.2) environment implemented in SystemVerilog. The verification strategy relies on constrained-random stimulus generation, protocol-level checking, and comprehensive functional coverage closure.

The top-level UVM testbench (overall_uvm) instantiates the complete top_veer_soc RTL design under test (DUT). The testbench architecture includes:

- **SPI UVM Agent:** Active/passive UVM verification IP monitoring spi_sck_o, spi_mosi_o, spi_miso_i, spi_ss_o. Includes a behavioral slave responder simulating external SPI flash memories.
- **I2C UVM Agent:** Active/passive UVM agent driving and monitoring scl_pad_i/o and sda_pad_i/o. Includes an I2C slave responder simulating EEPROM and sensor devices with configurable ACK/NACK responses.
- **UART UVM Agent:** Monitors uart_txd_o and drives uart_rxd_i, checking serial frame formats, start/stop bits, parity, and baud rate tolerances.
- **Interconnect Monitor:** Monitors 64-bit AHB-Lite transactions (HADDR, HWDATA, HRDATA, HRESP, HREADY) to verify bus protocol compliance and latency.
- **DMA Dedicated Slave Monitor:** Passive monitor tapping the dedicated 64-bit AXI4 slave port to verify DCCM address normalization and single-cycle transfer timing.
- **Self-Checking Scoreboards:** Independent scoreboards for DMA, SPI, I2C, and UART comparing transmitted data packets against golden expected payloads.

13.2 Functional Coverage Model (9 Covergroups)

Functional coverage is measured across nine comprehensive covergroups implemented in SystemVerilog, tracking all critical hardware states, corner cases, and protocol configurations:

- **1. dma_transfers_cg:** Tracks transfer lengths (1 to 64 KB), burst beat configurations (Single, 4, 8, 16), and mode transitions (M2M, M2P, P2M).
- **2. dma_channels_cg:** Monitors channel enabling, descriptor loading, and software abort sequences.
- **3. dccm_alignment_cg:** Validates 8-bit byte, 16-bit half-word, 32-bit word, and 64-bit double-word alignments in DCCM SRAM.
- **4. bus_contention_cg:** Crosses CPU LSU peripheral accesses with simultaneous DMA DCCM accesses to verify arbitration and lack of starvation.
- **5. spi_modes_cg:** Covers Modes 0, 1, 2, 3 (CPOL=0/1, CPHA=0/1), SCK dividers, and FIFO fill levels.
- **6. i2c_modes_cg:** Measures Standard (100k) and Fast (400k) speeds, Repeated START, STOP, ACK, and NACK conditions.
- **7. fifo_conditions_cg:** Covers FIFO empty, partially full, completely full, overrun, and underrun scenarios.
- **8. pic_interrupts_cg:** Verifies assertion, priority filtering, claiming, and clearing for all 5 PIC peripheral sources.
- **9. error_cases_cg:** Tracks response to illegal addresses, unaligned DMA pointers, slave ERROR responses, and timeout conditions.

13.3 Firmware Milestones & Hardware-Software Co-Verification

The SoC has been validated using a comprehensive firmware milestone suite (milestones 0x01 through 0x1D, and 0xAA). Firmware running natively on the VeeR EL2 core systematically verifies every architectural feature:

- Milestones 0x01 to 0x05 validate CPU core execution, DCCM SRAM access, Boot ROM fetching, and CSR programming.
- Milestones 0x06 to 0x0A validate AHB interconnect arbitration, address decoding, and peripheral CSR reads/writes.
- Milestones 0x0B to 0x10 validate DMA Memory-to-Memory transfers, dedicated 64-bit DMA slave port access, and alignment checks.

- Milestones 0x11 to 0x15 validate SPI master operation, FIFO pushes/pops, Mode 0-3 transfers, and DMA-accelerated SPI streaming.
- Milestones 0x16 to 0x19 validate I2C master prescaling, START/STOP sequencing, slave ACK detection, and DMA-accelerated I2C transfers.
- Milestones 0x1A to 0x1D validate UART 16550 baud divisor generation, 8N1 serial framing, and PIC external interrupt servicing.
- Milestone 0xAA executes an integrated stress test concurrently exercising CPU execution, DMA transfers, SPI transmission, and I2C reception.

13.4 Regression Test Matrix & Functional Verification Closure

Table 13.1 provides the complete regression results across the 10 dedicated UVM test suites executed via regression.sh. All tests report 100% pass rates, zero UVM errors, and complete functional coverage closure.

Test Suite Identifier	Target Architectural Feature	Cycles	UVM Errors	Scoreboard Matches	Functional Status
soc_overall_test	Full SoC integrated boot, milestones 0x01-0x1D/0xAA, DMA/SPI/I2C/UART	1,850,000	0	100% Match (0 Mismatch)	PASSED (100%)
soc_dma_m2m_test	DMA memory-to-memory dedicated 64b slave port DCCM transfers	420,000	0	100% Match (16/16)	PASSED (100%)
soc_spi_tx_test	DMA-accelerated SPI transmission, 16-deep FIFO streaming, Mode 0-3	650,000	0	100% Match (64/64 B)	PASSED (100%)
soc_spi_rx_test	DMA-accelerated SPI reception, slave responder data capture into DCCM	680,000	0	100% Match (64/64 B)	PASSED (100%)
soc_i2c_tx_test	DMA-accelerated I2C multi-byte transmission, 100k/400k baud prescaler	890,000	0	100% Match (32/32 B)	PASSED (100%)
soc_i2c_rx_test	DMA-accelerated I2C reception, open-drain SDA sampling into DCCM	910,000	0	100% Match (32/32 B)	PASSED (100%)

Test Suite Identifier	Target Architectural Feature	Cycles	UVM Errors	Scoreboard Matches	Functional Status
soc_uart_tx_test	16550 UART serial diagnostic transmission, baud divisor checking	350,000	0	100% Match (128/128 B)	PASSED (100%)
soc_dma_error_test	DMA bus error injection, unaligned address traps, PIC IRQ Source 2	280,000	0	Error Handled Cleanly	PASSED (100%)
soc_bus_contention_test	Simultaneous CPU LSU AHB access and DMA dedicated DCCM transfers	740,000	0	Zero Pipeline Starvation	PASSED (100%)
soc_random_dma_test	Constrained-random DMA payload sizes, burst beats, and address offsets	1,200,000	0	100% Match (120/120)	PASSED (100%)

14. Design Constraints and Limitations

To ensure reliable deployment in synthesized ASIC or FPGA implementations, the following design constraints and operational limitations must be observed:

- **Bus Bandwidth:** The 64-bit AHB-Lite interconnect accommodates a theoretical maximum bandwidth of 400 MB/s at 50 MHz. While the DMA dedicated slave port offloads DCCM traffic, concurrent peripheral accesses share the AHB fabric with CPU LSU transactions.
- **Memory Alignment:** 64-bit transfers targeting DCCM SRAM must be aligned to 8-byte address boundaries (addr[2:0] == 3'b000). Attempting unaligned 64-bit accesses results in an immediate DMA fault and assertion of the dma_error interrupt.
- **FIFO Thresholds:** The SPI controller FIFOs are 16 entries deep (32-bit width). DMA burst transfers must not exceed 16 beats per burst to prevent FIFO overflow when transmitting faster than the physical serial shift rate.
- **Channel Concurrency:** The DMA controller processes one active transfer channel at a time. Multi-channel queueing must be managed via software descriptors in memory.
- **Clocking Constraints:** The design requires that all modules share the single 50 MHz synchronous clock domain. External asynchronous interfaces (e.g., UART RX, I2C SDA/SCL) are synchronized via on-chip double-flop synchronizers.

15. Conclusion

15.1 Summary of SoC Architectural Achievements

The VeeR EL2-Based Communication SoC successfully realizes an advanced, low-power, high-throughput microcontroller architecture tailored for serial communication acceleration. By pairing the Western Digital VeeR EL2

RV32IMC processor core with a dedicated 64-bit AXI4 DMA slave interface and an AMBA AHB-Lite interconnect, the SoC achieves:

- Elimination of CPU PIO polling overhead during high-speed SPI and multi-byte I2C transfers.
- Direct, single-cycle SRAM access between the DMA controller and the core DCCM memory, achieving up to 400 MB/s memory transfer rates.
- Complete architectural segregation between peripheral control traffic (AHB-Lite) and bulk memory data traffic (dedicated core DMA port).
- Low-latency, deterministic interrupt response via the core-integrated 15-priority Programmable Interrupt Controller.

15.2 Verification Closure & Readiness

The design has achieved 100% verification closure across all 30 firmware milestones and all 10 UVM regression test suites, reporting zero protocol mismatches, zero UVM errors, and complete functional coverage. The RTL implementation is fully validated, robust, and production-ready for FPGA prototyping or ASIC tapeout.